

优先级队列

概述：需求与动机

循优先级访问

❖ 应用举例

- 离散事件模拟
- 操作系统：任务调度/中断处理/MRU/...
- 输入法：词频调整

❖ 极值元素：须反复地、快速地定位

集合组成：可动态变化

元素优先级：可动态变化

❖ 作为底层数据结构所支持的高效操作

是很多高效算法的基础

- 内部、外部、在线排序
- 贪心算法：Huffman编码、Kruskal
- 平面扫描算法中的事件队列
- ...

优先级队列

❖ `template <typename T> struct PQ { //priority queue`

```
virtual void insert( T ) = 0;
```

```
virtual T getMax() = 0;
```

```
virtual T delMax() = 0;
```

`};` //作为ADT的PQ有多种实现方式, 各自的效率及适用场合也不尽相同

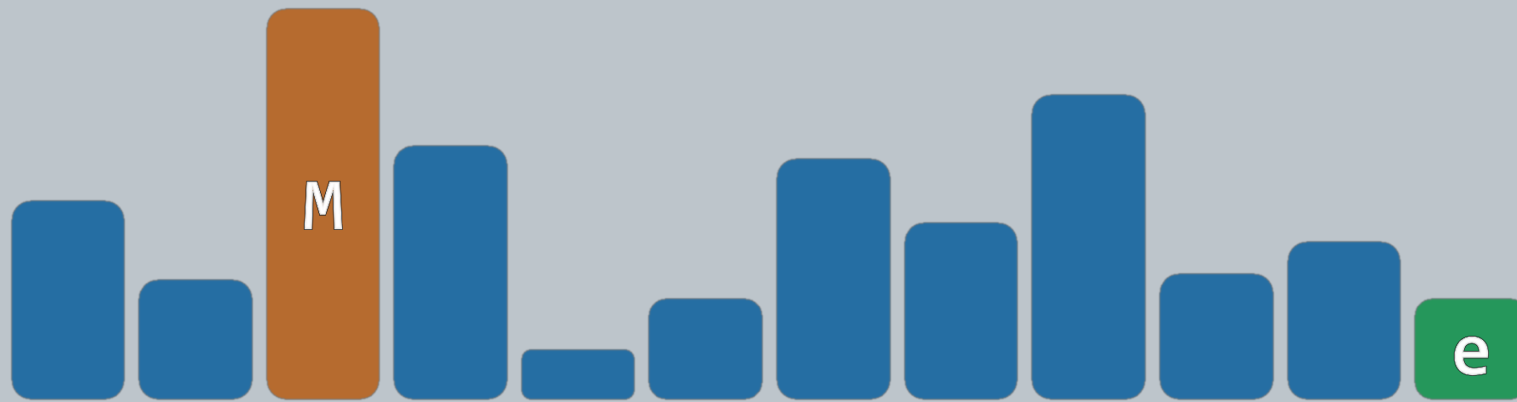
❖ Stack和Queue, 都是PQ的特例——优先级完全取决于元素的**插入次序**

❖ Steap和Queap, 也是PQ的特例——插入和删除的**位置受限**

优先级队列

概述：基本实现

Vector



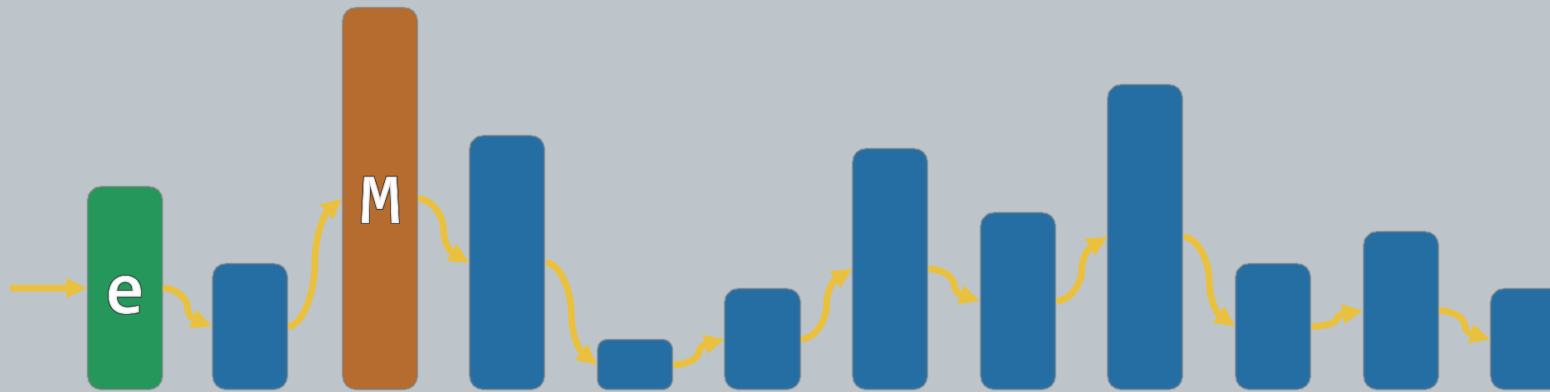
<code>getMax()</code>	<code>delMax()</code>	<code>insert()</code>
<code>traverse()</code> $\Theta(n)$	<code>remove(traverse())</code> $\Theta(n) + \mathcal{O}(n) = \Theta(n)$	<code>insertAsLast(e)</code> $\mathcal{O}(1)$

Sorted Vector



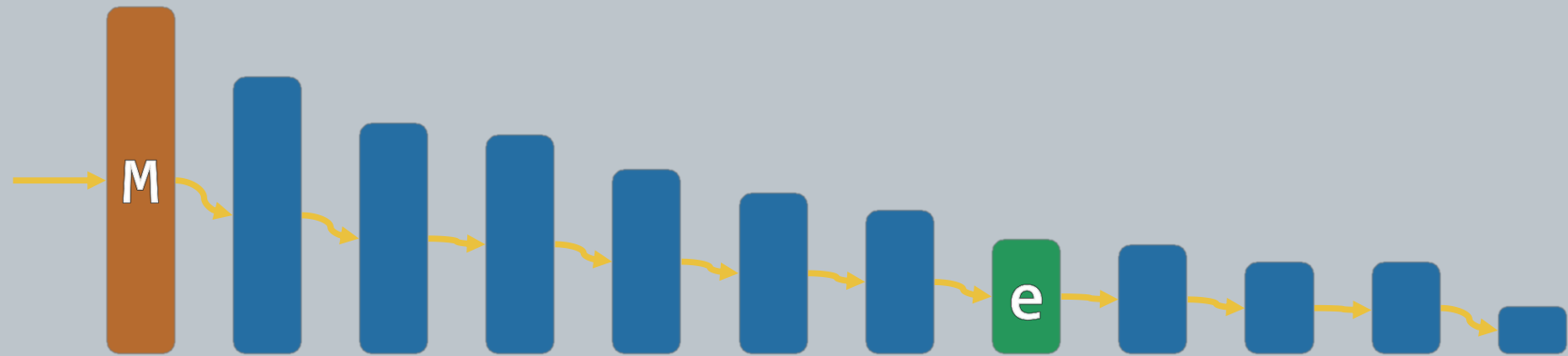
getMax()	delMax()	insert()
$[n - 1]$ $O(1)$	remove($n - 1$) $O(1)$	insert($1 + \text{search}(e)$, e) $O(\log n) + O(n) = O(n)$

List



getMax()	delMax()	insert()
traverse() $\Theta(n)$	remove(traverse()) $\Theta(n) + \mathcal{O}(1) = \Theta(n)$	insertAsFirst(e) $\mathcal{O}(1)$

Sorted List



<code>getMax()</code>	<code>delMax()</code>	<code>insert()</code>
<code>first()</code> $O(1)$	<code>remove(first())</code> $O(1)$	<code>insertA(search(e), e)</code> $O(n) + O(1) = O(n)$

BBST

❖ AVL、Splay、Red-black: 三个接口均只需 $\mathcal{O}(\log n)$ 时间

但是, BBST的功能远远超出了PQ的需求...

$$\text{❖ PQ} = \boxed{1 \times \text{insert}()} + \boxed{0.5 \times \text{search}()} + \boxed{0.5 \times \text{remove}()}$$

❖ 若只需查找极值元, 则不必维护所有元素之间的全序关系, 偏序足矣

❖ 因此有理由相信, 存在某种更为简单、维护成本更低的实现方式

使得各功能接口的时间复杂度依然为 $\mathcal{O}(\log n)$, 而且实际效率更高

❖ 当然, 就最坏情况而言, 这类实现方式已属最优——为什么?

优先级队列
完全二叉堆：结构

结构性：逻辑元素、物理节点依层次遍历次序彼此对应

```
#define Parent(i) ((i) >> 1)
```

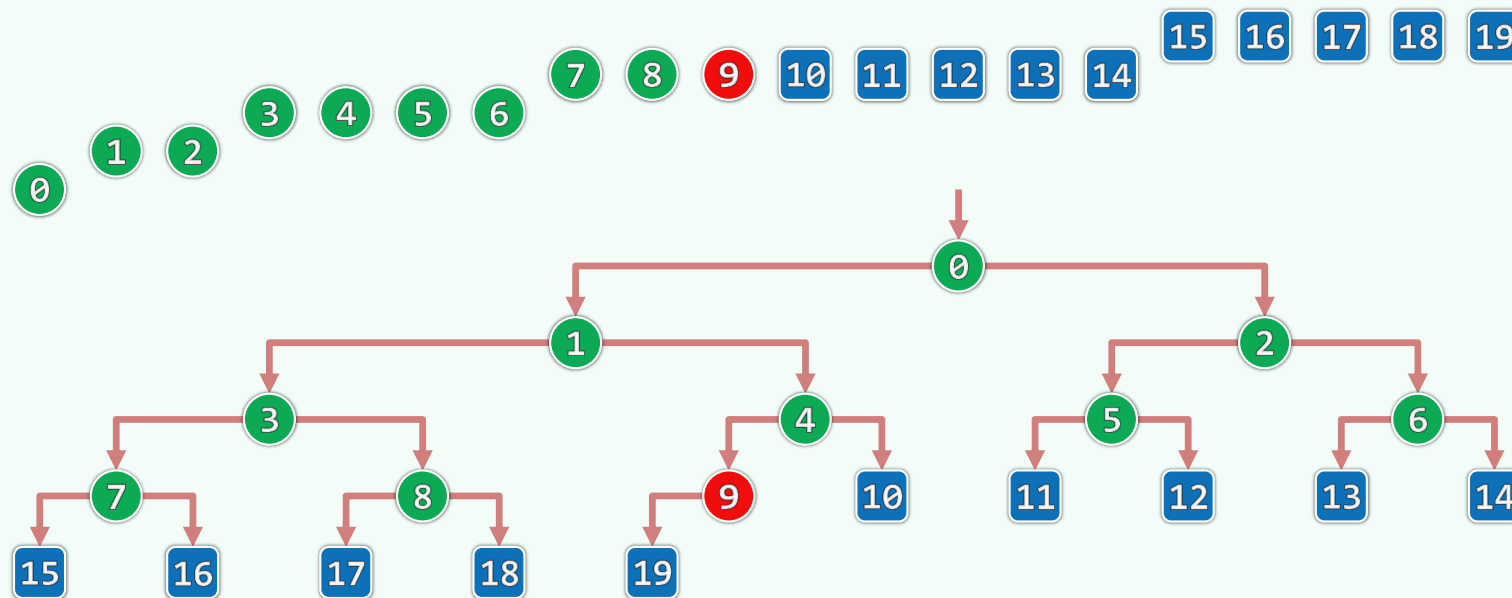
❖ 逻辑上，等同于完全二叉树

```
#define LChild(i) (1 + ((i) << 1))
```

物理上，直接借助向量实现

```
#define RChild(i) ((1 + (i)) << 1)
```

❖ 内部节点的最大秩 = $\lfloor \frac{n-2}{2} \rfloor = \lceil \frac{n-3}{2} \rceil$



PQ_ComplHeap = PQ + Vector

```
❖ template <typename T> struct PQ_ComplHeap : public PQ<T>, public Vector<T> {  
  
    PQ_ComplHeap( T* A, Rank n ) { copyFrom( A, 0, n ); heapify( _elem, n ); }  
  
    void insert( T ); T getMax(); T delMax();  
  
};
```

```
❖ template <typename T> Rank percolateDown( T* A, Rank n, Rank i ); //下滤
```

```
❖ template <typename T> Rank percolateUp( T* A, Rank i ); //上滤
```

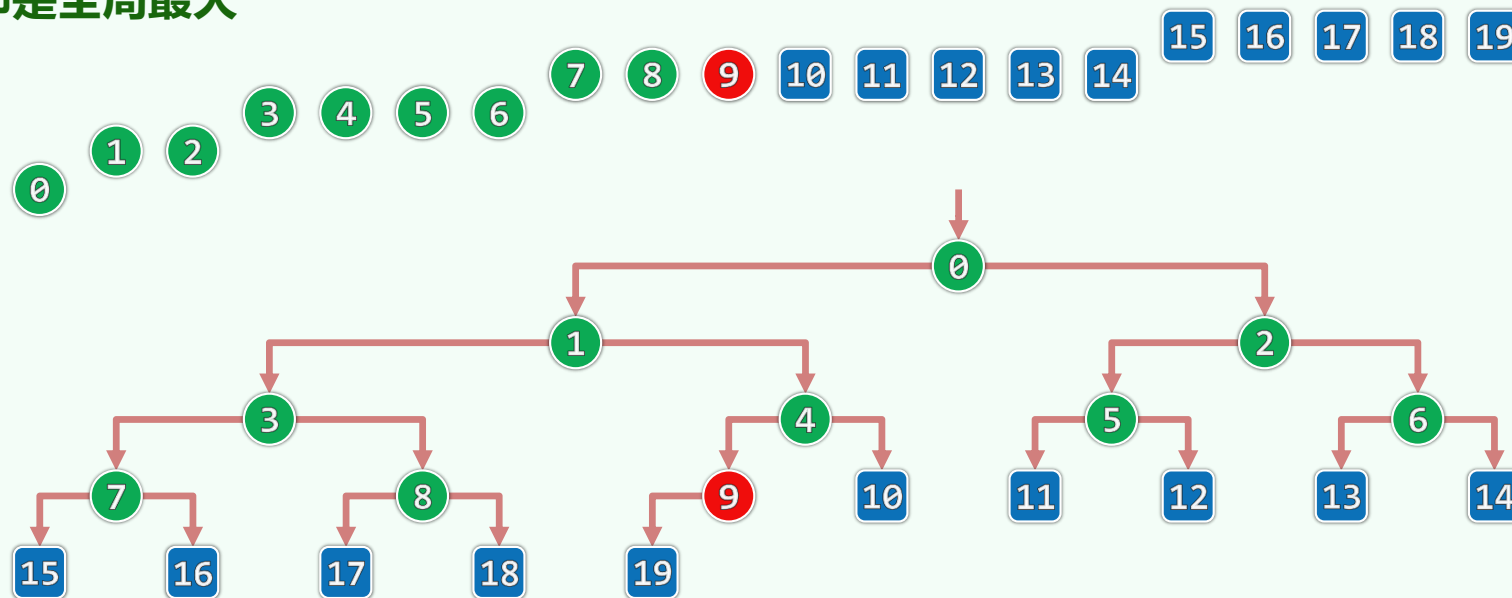
```
❖ template <typename T> void heapify( T* A, Rank n); //Floyd建堆算法
```

堆序性

❖ `template <typename T> T PQ_ComplHeap<T>::getMax() { return _elem[0]; }`

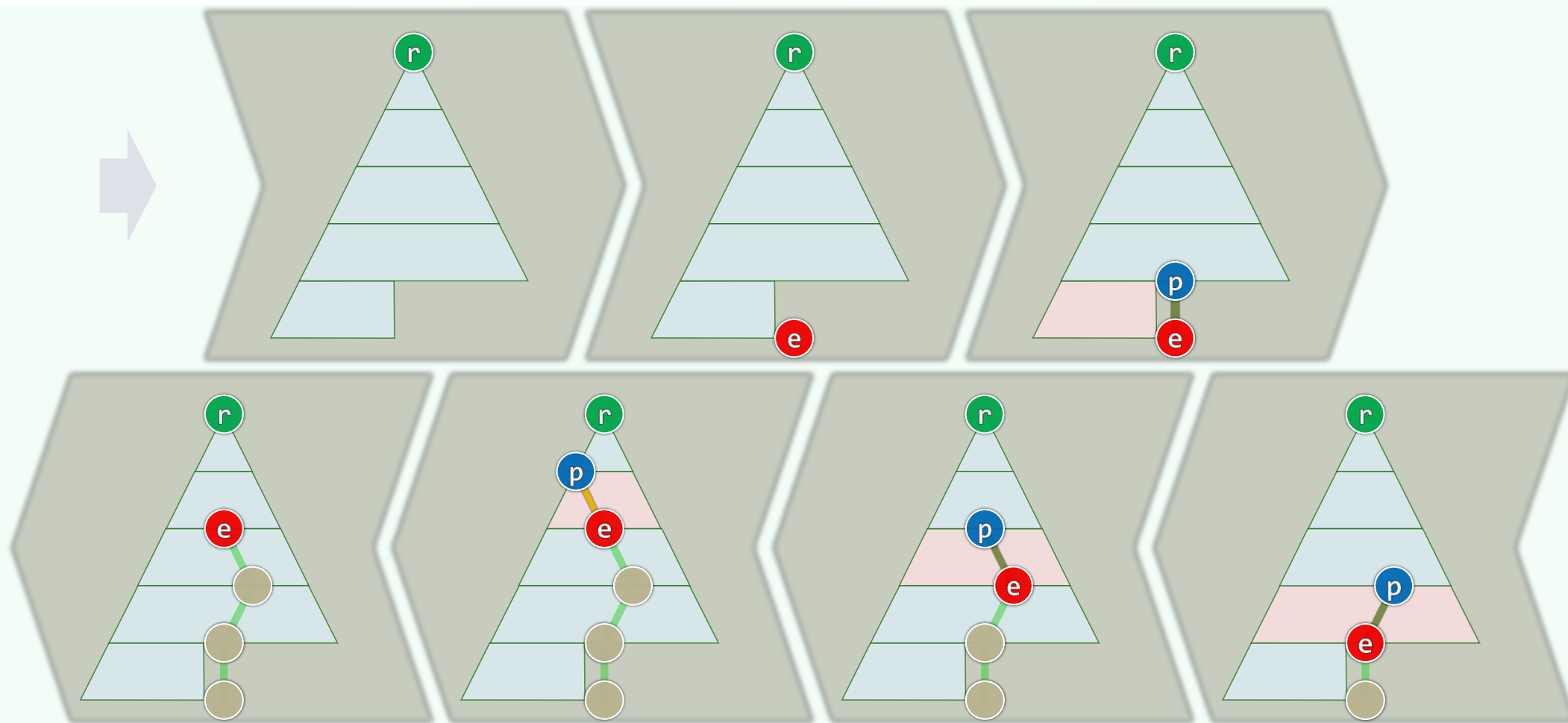
❖ 只要 $0 < i$, 必满足 $H[i] \leq H[\text{Parent}(i)]$

故 $H[0]$ 即是全局最大

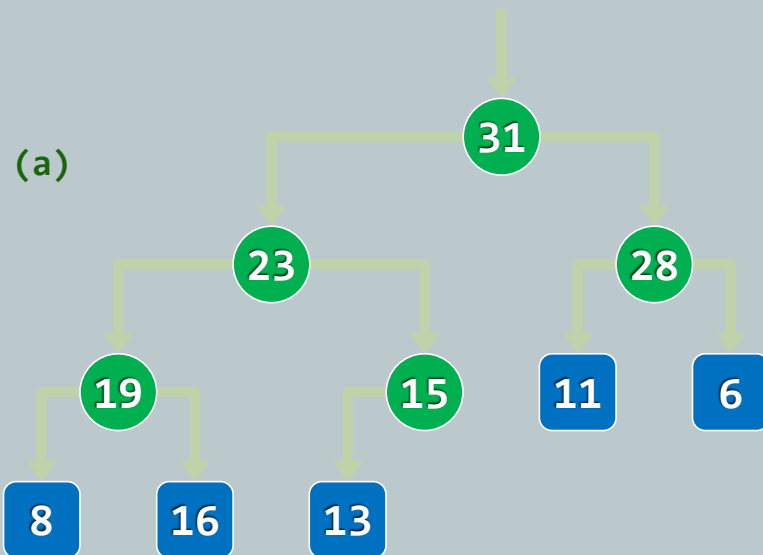


优先级队列
完全二叉堆：插入

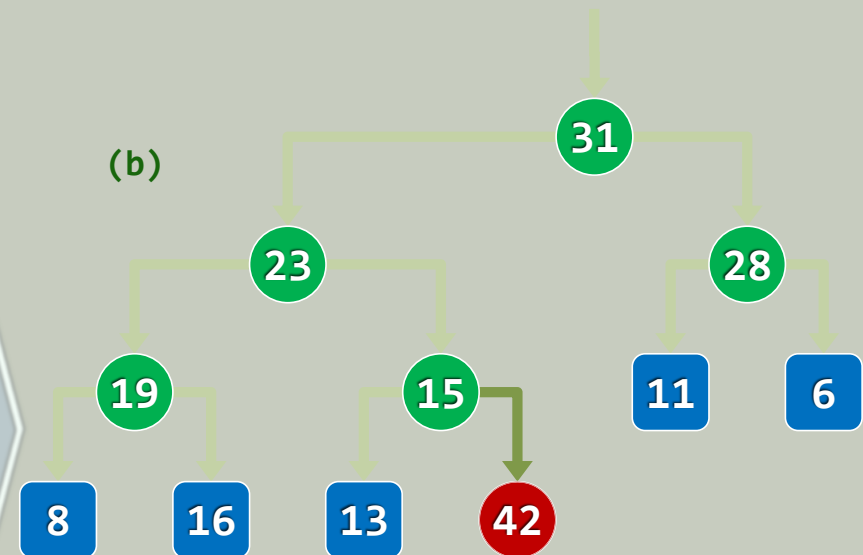
算法：逐层上滤



实例 (1/5)

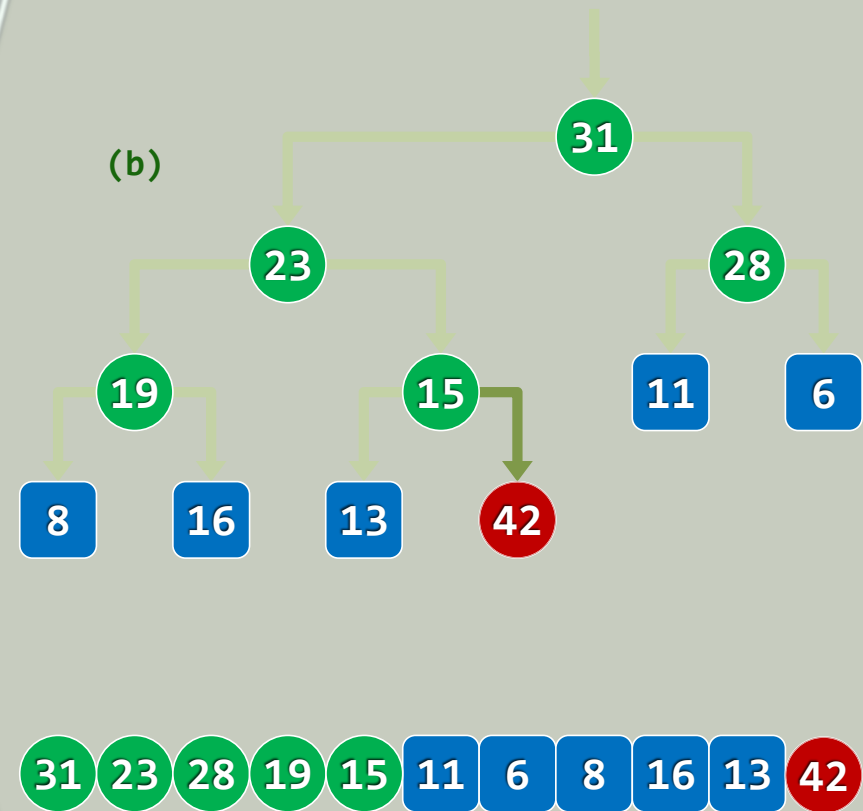
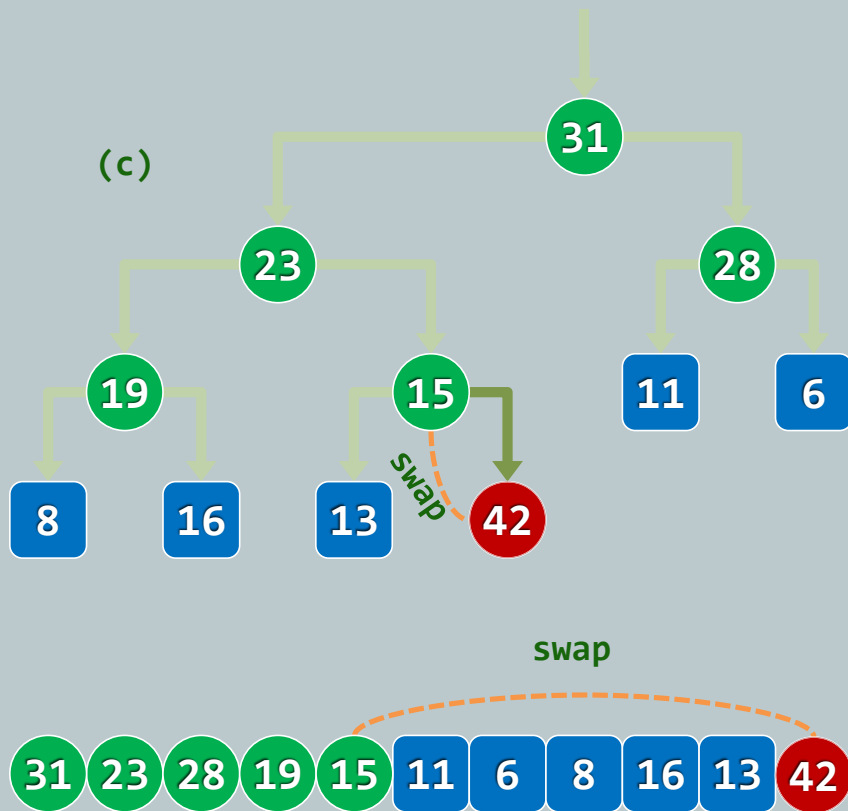


31 23 28 19 15 11 6 8 16 13

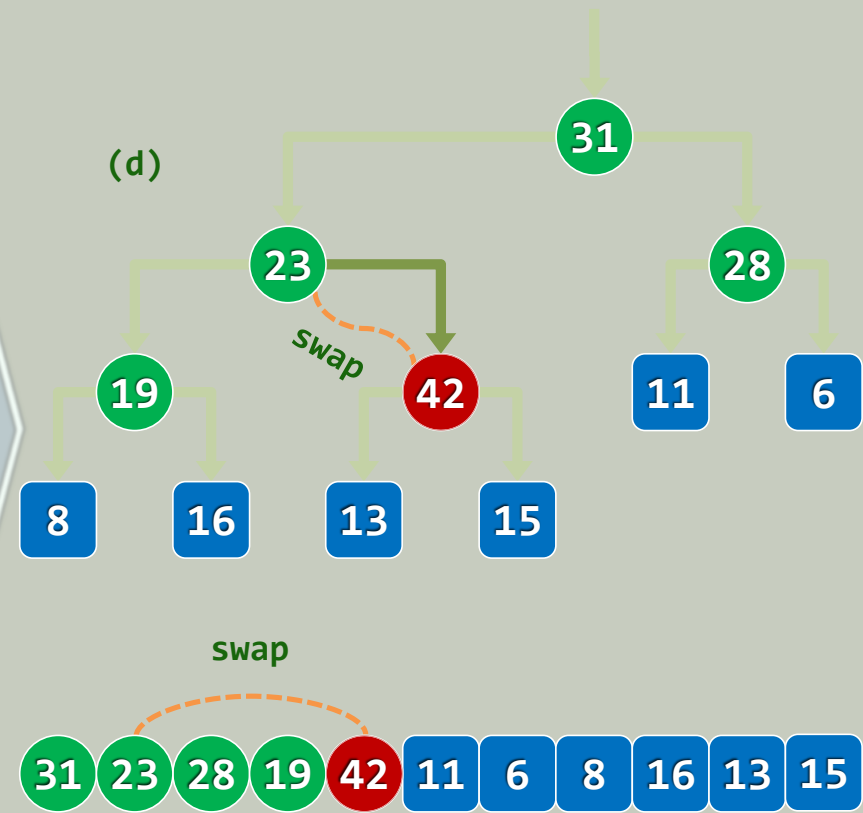
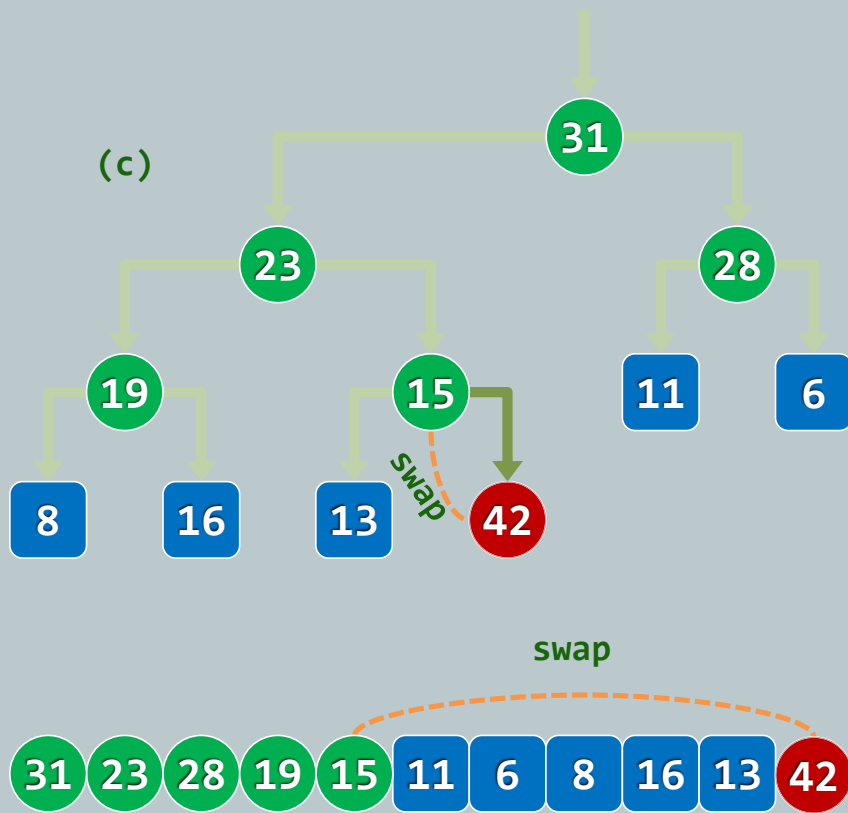


31 23 28 19 15 11 6 8 16 13 42

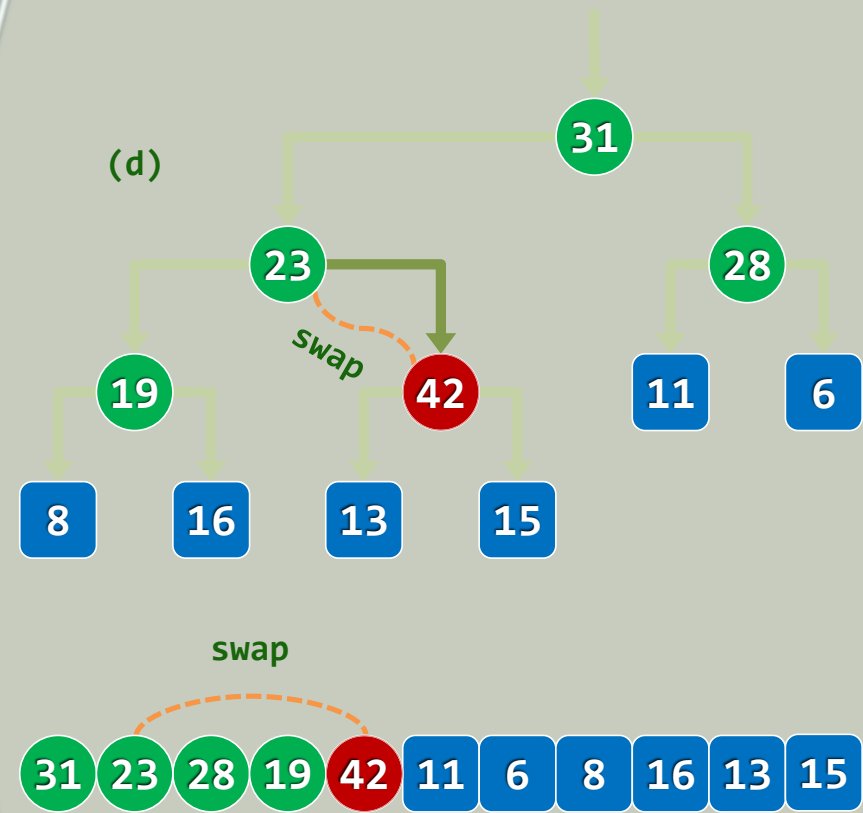
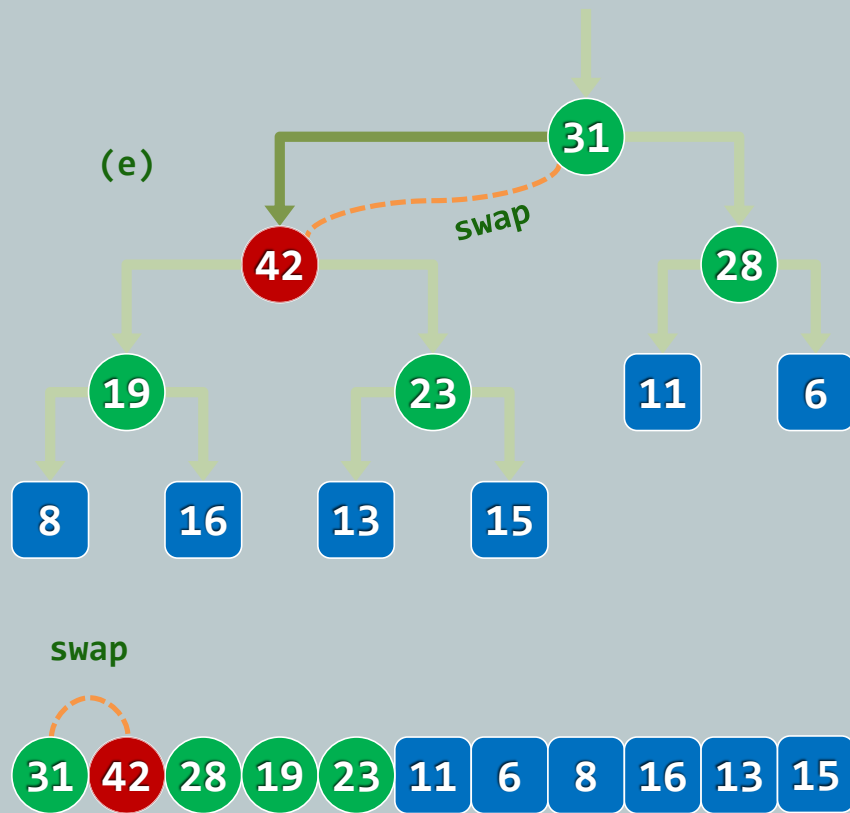
实例 (2/5)



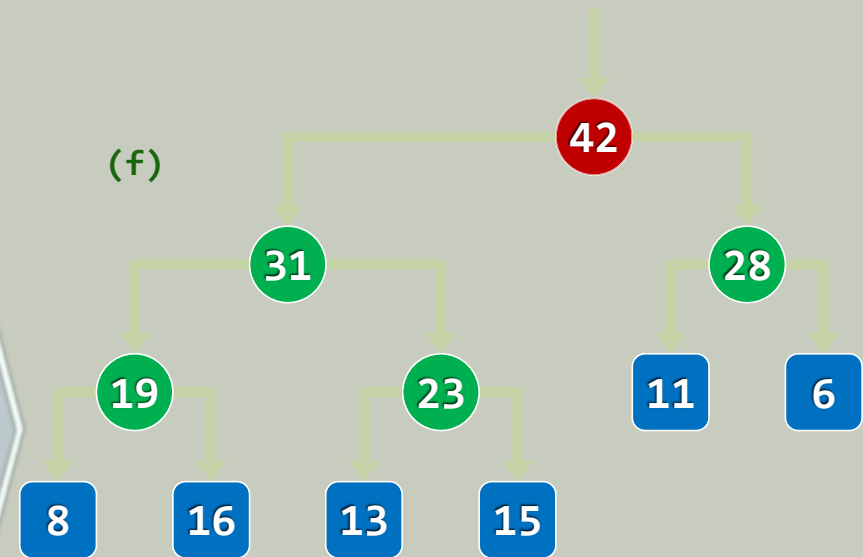
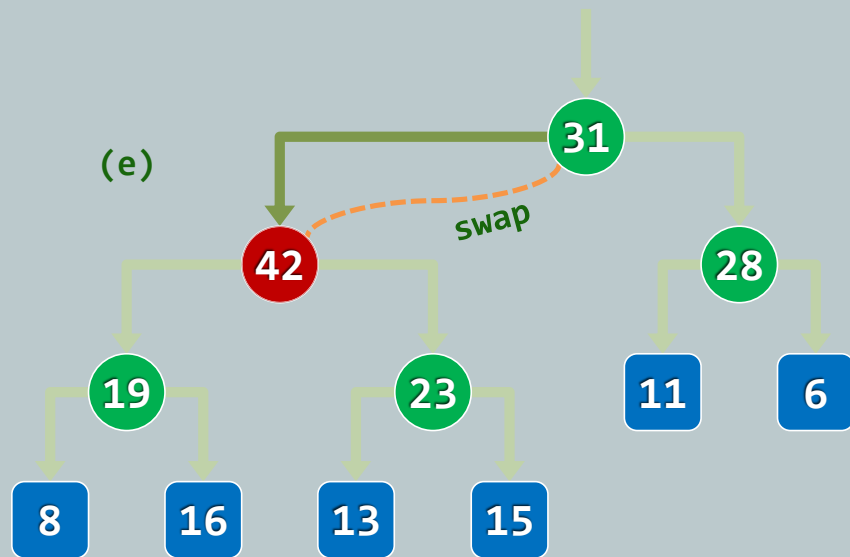
实例 (3/5)



实例 (4/5)



实例 (5/5)



实现

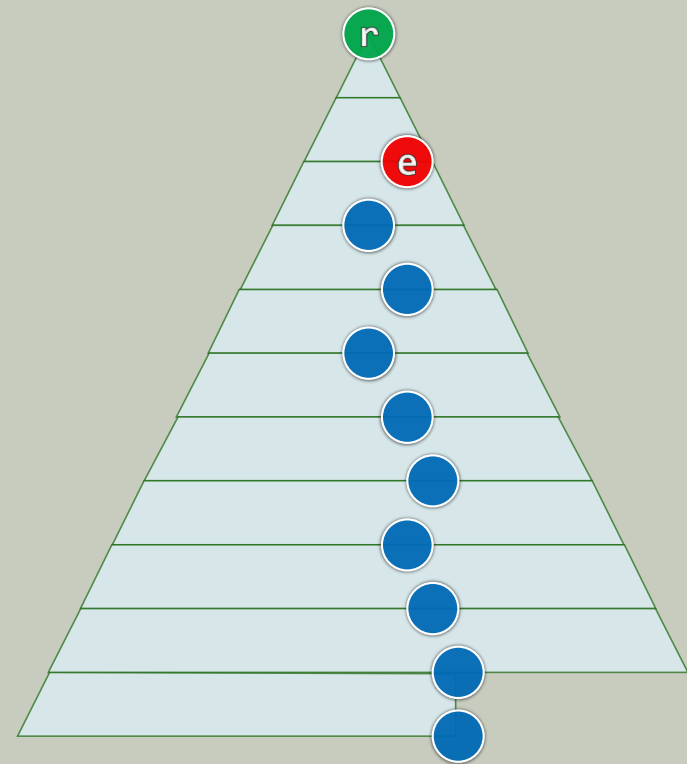
```
❖ template <typename T> void PQ_ComplHeap<T>::insert( T e ) //插入
    { Vector<T>::insert( e ); percolateUp( _elem, _size - 1 ); } //此insert()非彼

❖ template <typename T> Rank percolateUp( T* A, Rank i ) { //0 <= i < _size
    while ( 0 < i ) { //在抵达堆顶之前, 反复地
        Rank j = Parent( i ); //考查[i]之父亲[j]
        if ( lt( A[i], A[j] ) ) break; //一旦父子顺序, 上滤旋即完成; 否则
        swap( A[i], A[j] ); i = j; //父子换位, 并继续考查上一层
    } //while
    return i; //返回上滤最终抵达的位置
}
```

效率

- ❖ e在上滤过程中，只可能与祖先们交换
- ❖ 完全树必平衡，e的祖先不超过 $O(\log n)$ 个
- ❖ 故知插入操作可在 $O(\log n)$ 时间内完成
- ❖ 然而就数学期望而言

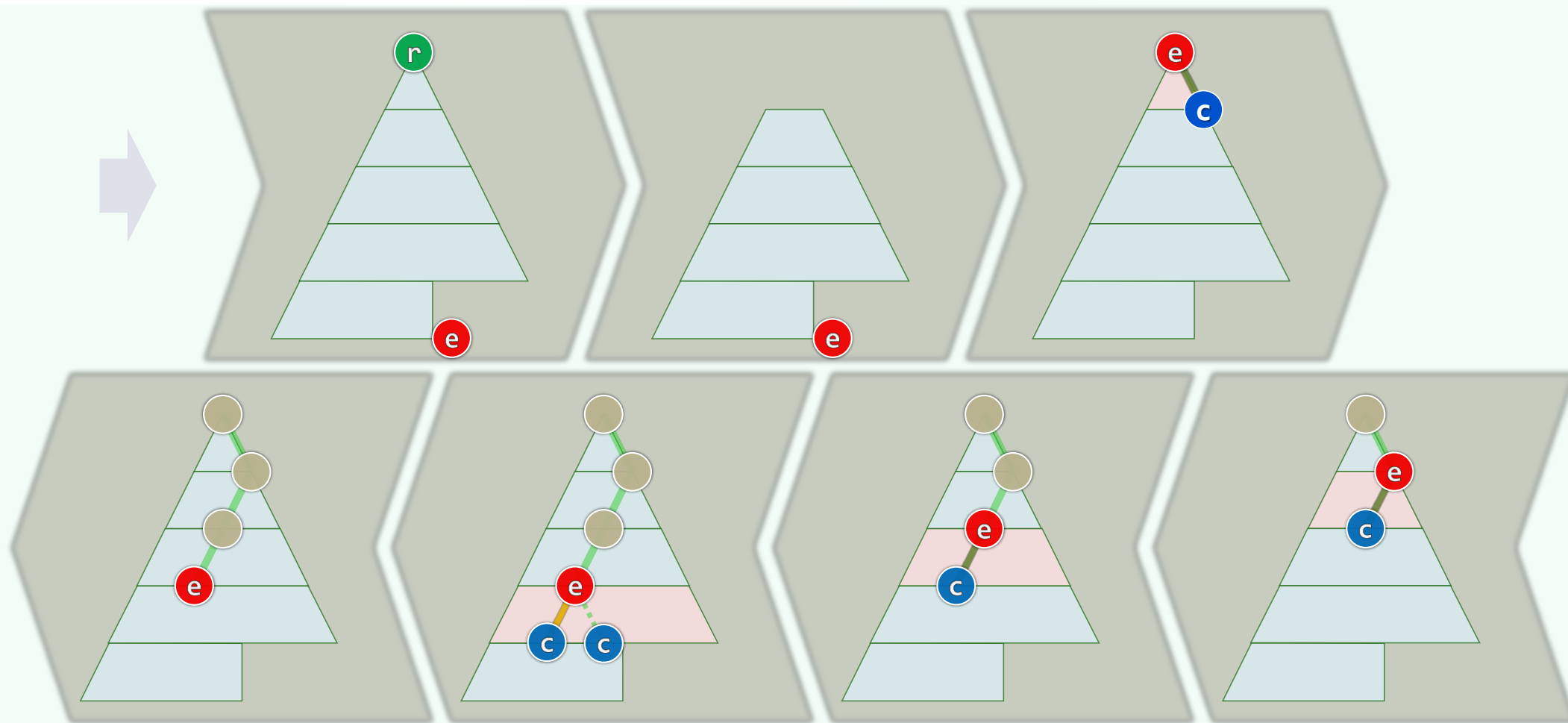
实际效率往往远远更高...



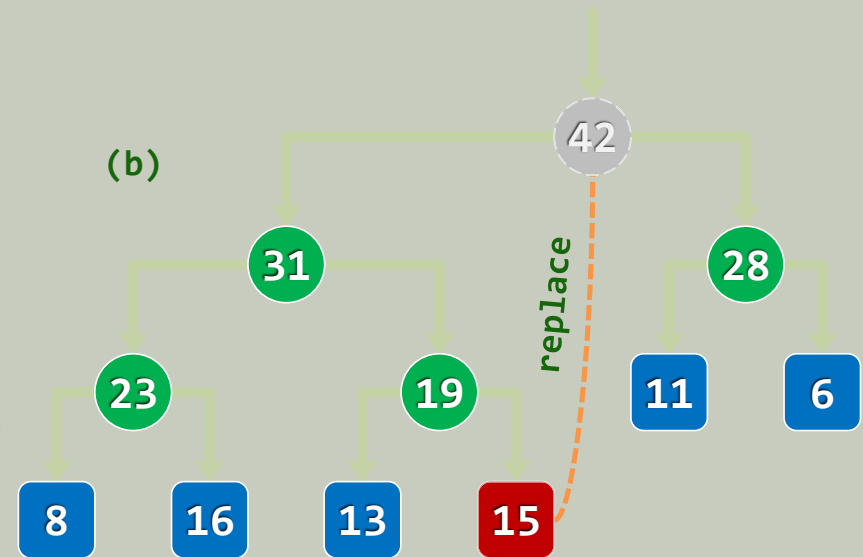
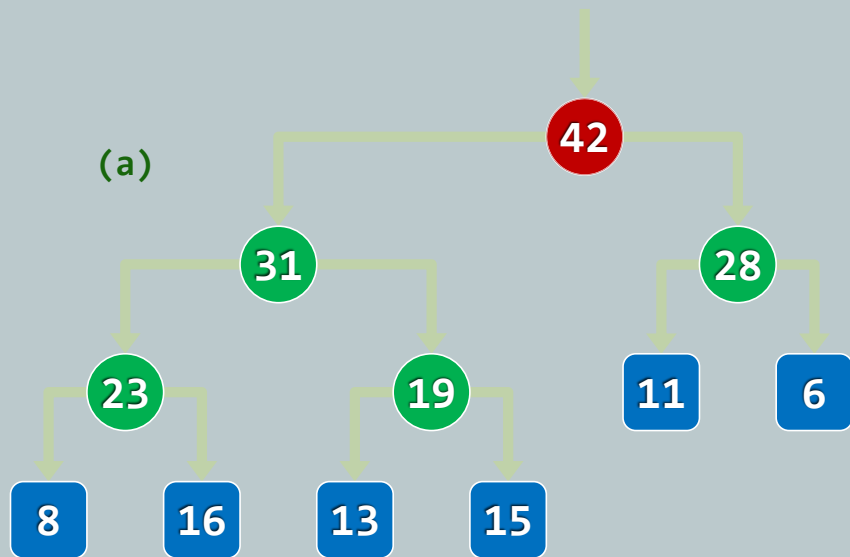
优先级队列

完全二叉堆：删除

算法：割肉补疮 + 逐层下滤



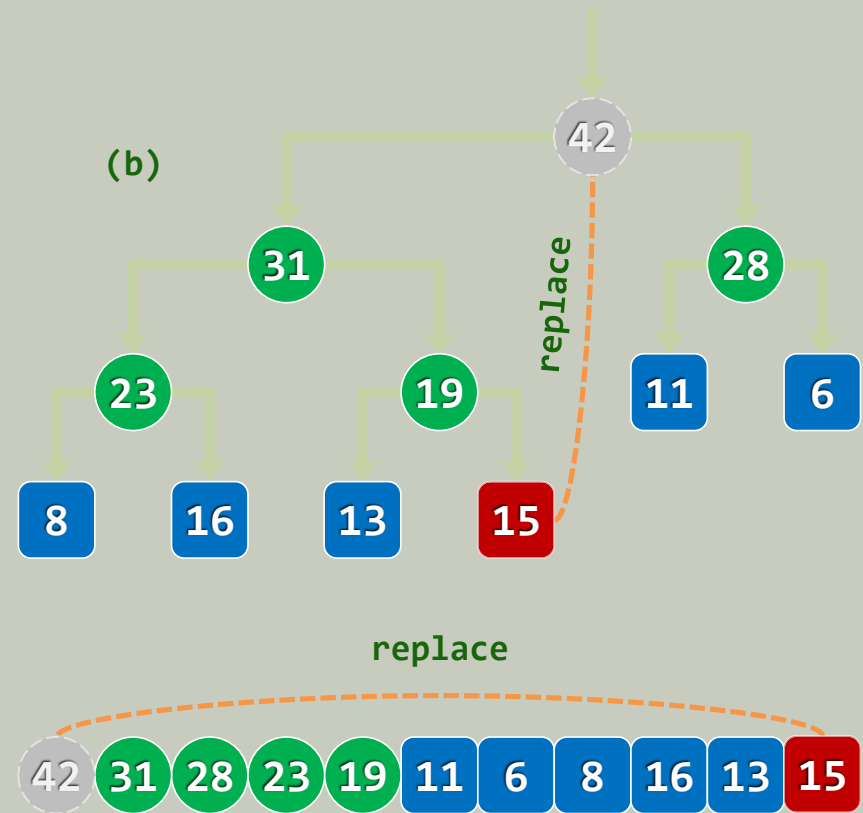
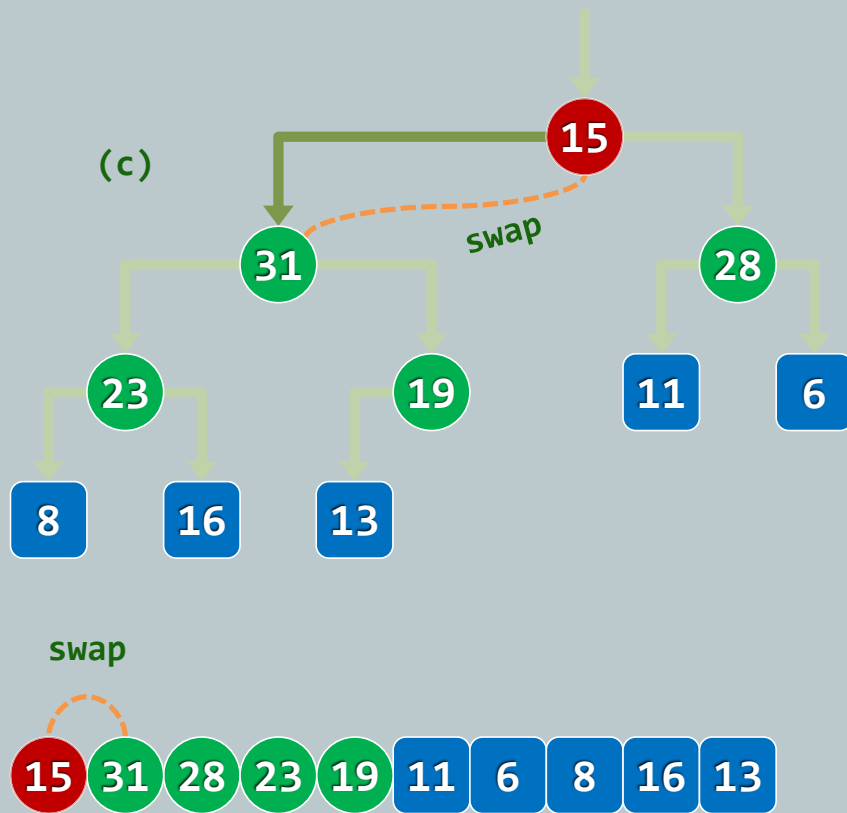
实例 (1/5)



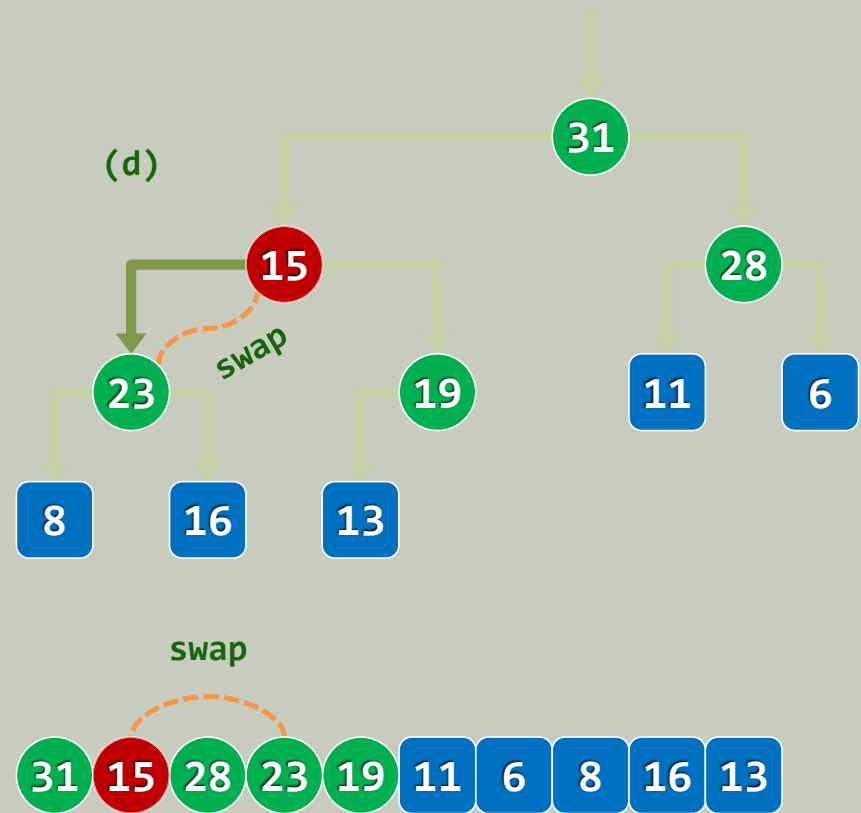
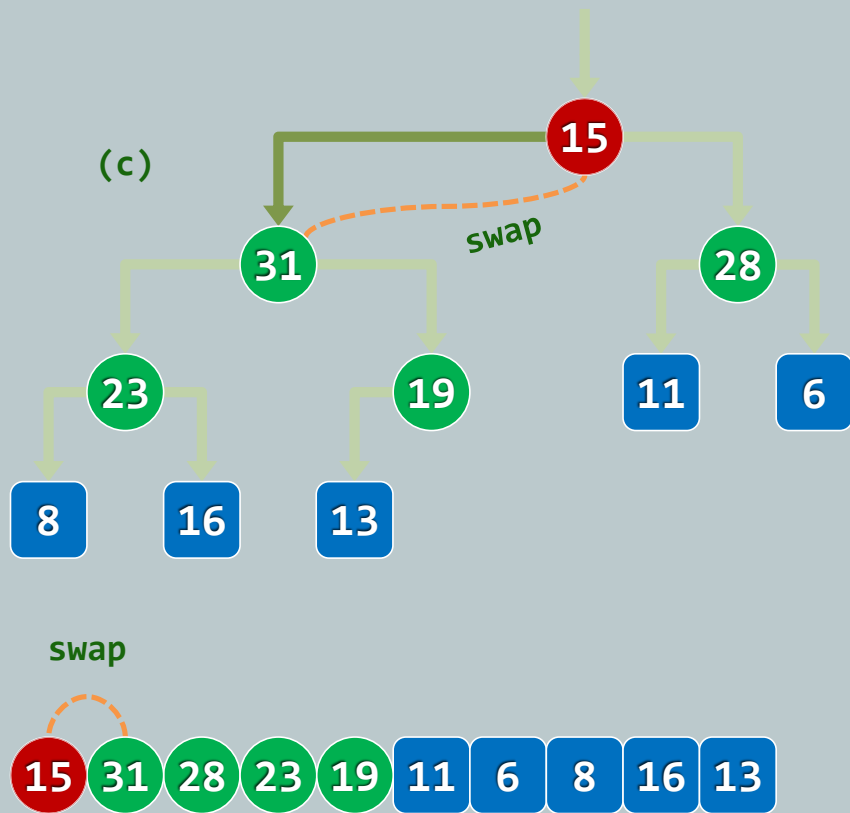
replace



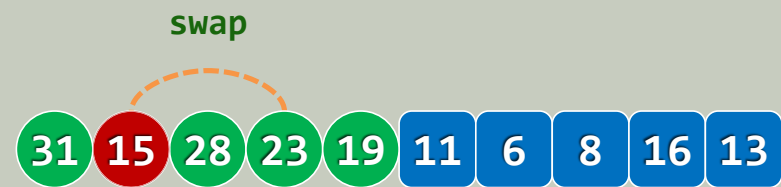
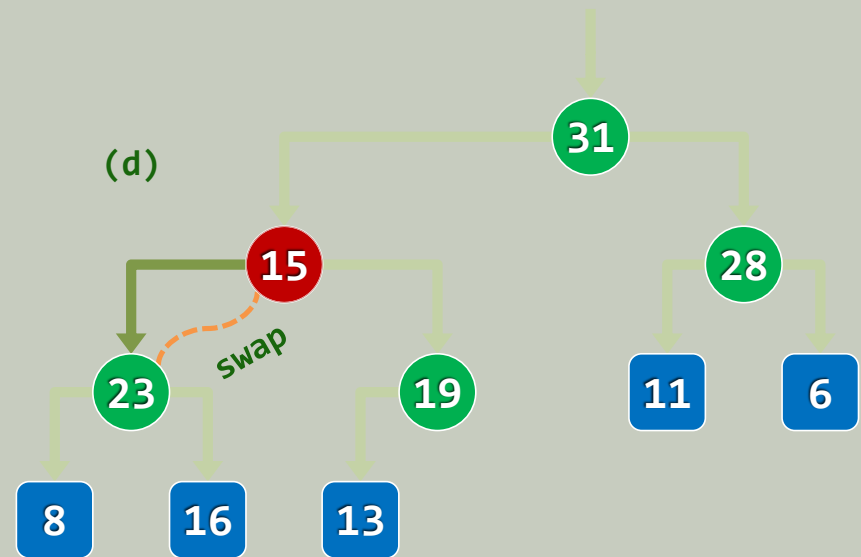
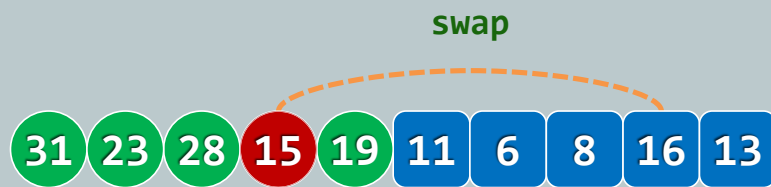
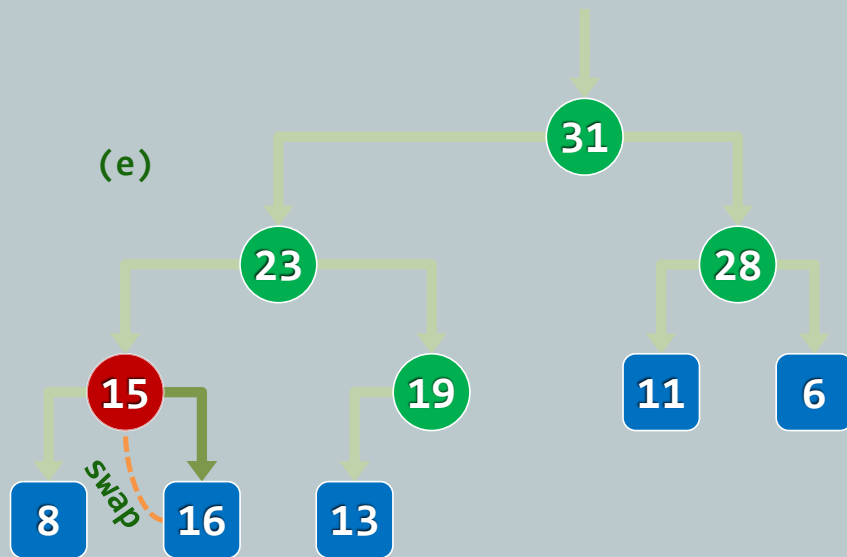
实例 (2/5)



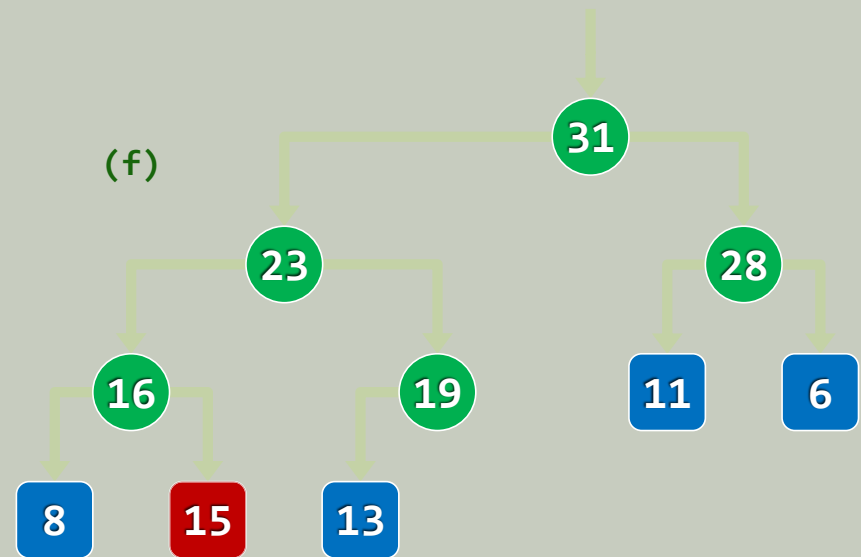
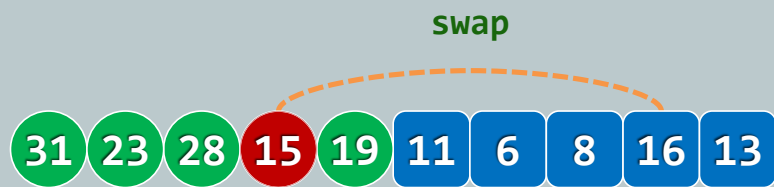
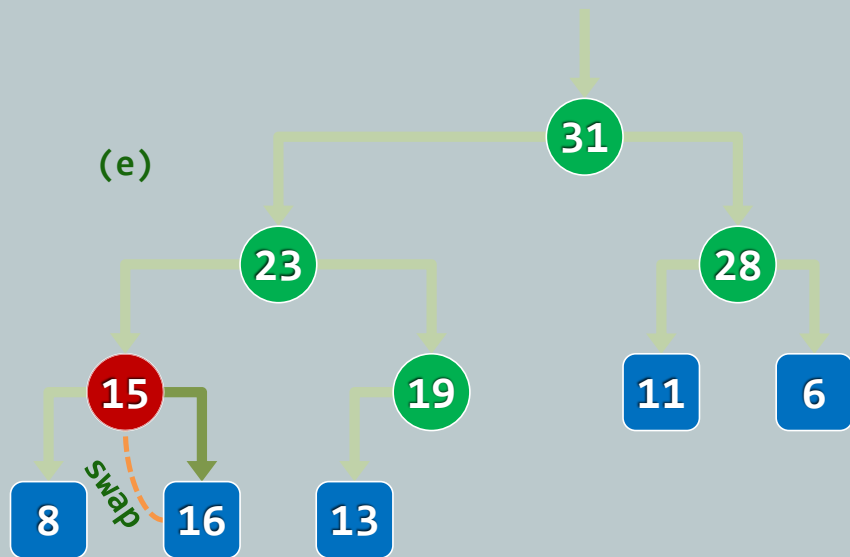
实例 (3/5)



实例 (4/5)



实例 (5/5)



实现

```
❖ template <typename T> T PQ_ComplHeap<T>::delMax() { //删除
    T maxElem = _elem[0]; _elem[0] = _elem[ --_size ]; //摘除堆顶, 代之以末词条
    percolateDown( _elem, _size, 0 ); //对新堆顶实施下滤
    return maxElem; //返回此前备份的最大词条
}

❖ template <typename T> Rank percolateDown( T* A, Rank n, Rank i ) { //0 <= i < n
    Rank j; //i及其 (至多两个) 孩子中, 堪为父者
    while ( i != ( j = ProperParent( A, n, i ) ) ) //只要i非j, 则
        { swap( A[i], A[j] ); i = j; } //换位, 并继续考察i
    return i; //返回下滤抵达的位置 (亦i亦j)
}
```

效率

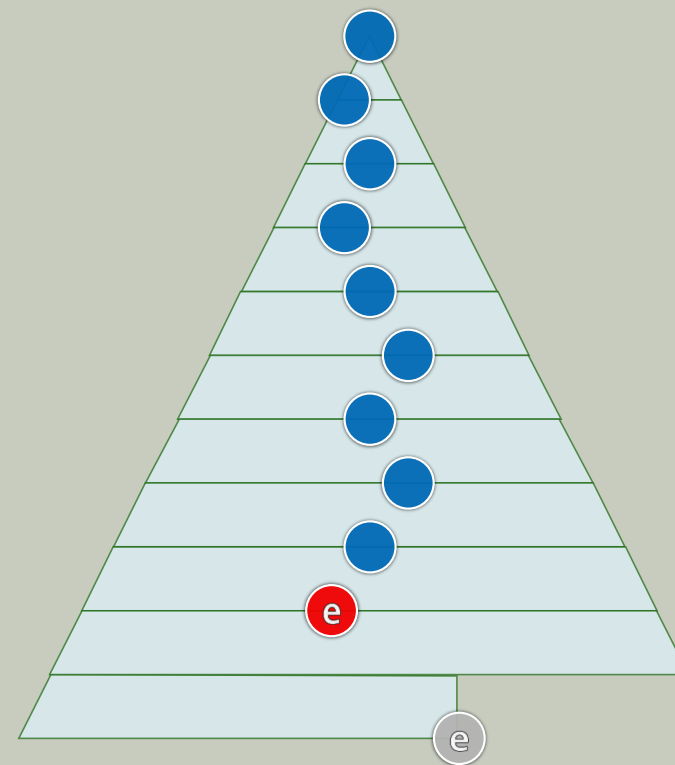
❖ e在每一高度至多交换一次

累计交换不超过 $O(\log n)$ 次

❖ 通过下滤，可在 $O(\log n)$ 时间内

- 删除堆顶节点，并
- 整体重新调整为堆

❖ 数学期望呢？



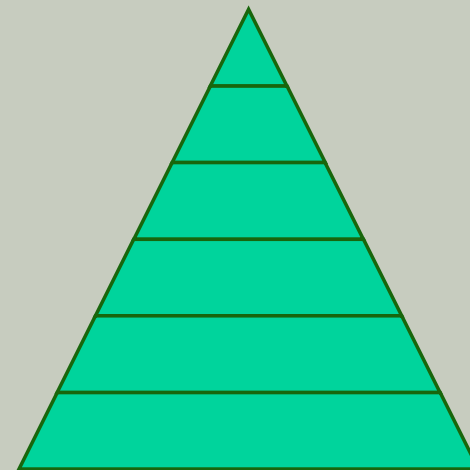
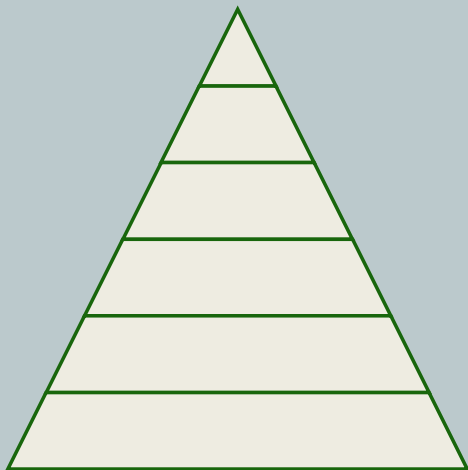
优先级队列

完全二叉堆：批量建堆

自上而下的上滤 (1/2)

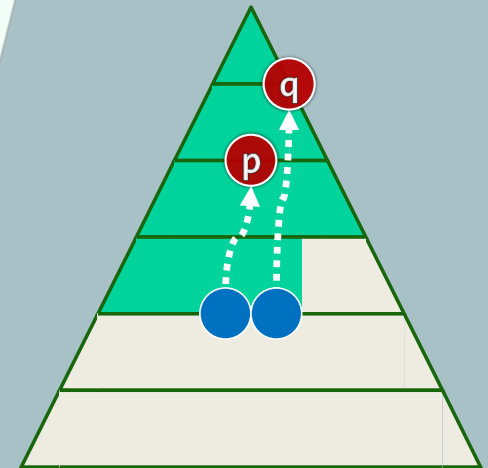
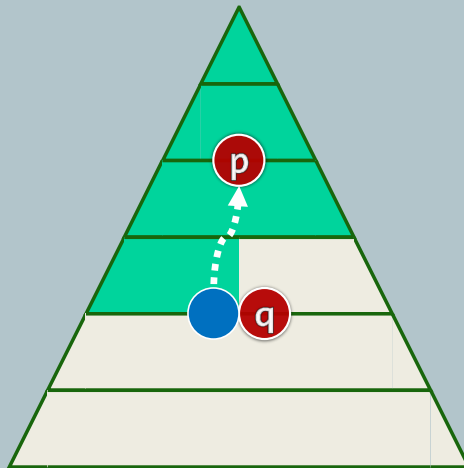
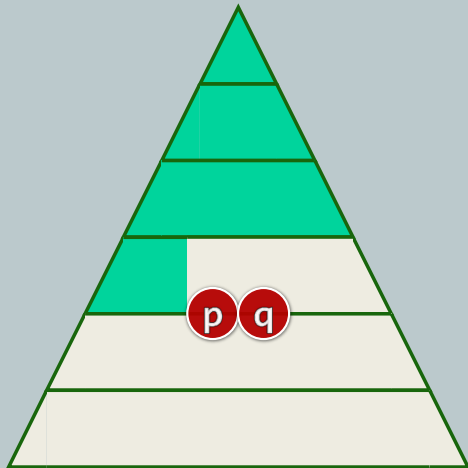
```
PQ_ComplHeap( T* A, Rank n )
```

```
{ copyFrom( A, 0, n ); heapify( _elem, n ); }
```



自上而下的上滤 (2/2)

```
template <typename T> void heapify( T* A, const Rank n ) { //蛮力  
    for ( Rank i = 1; i < n; i++ ) //按照逐层遍历次序逐一  
        percolateUp( A, i ); //经上滤插入各节点  
}
```



效率

❖ 最坏情况下

- 每个节点都需上滤至根
- 所需成本线性正比于其深度

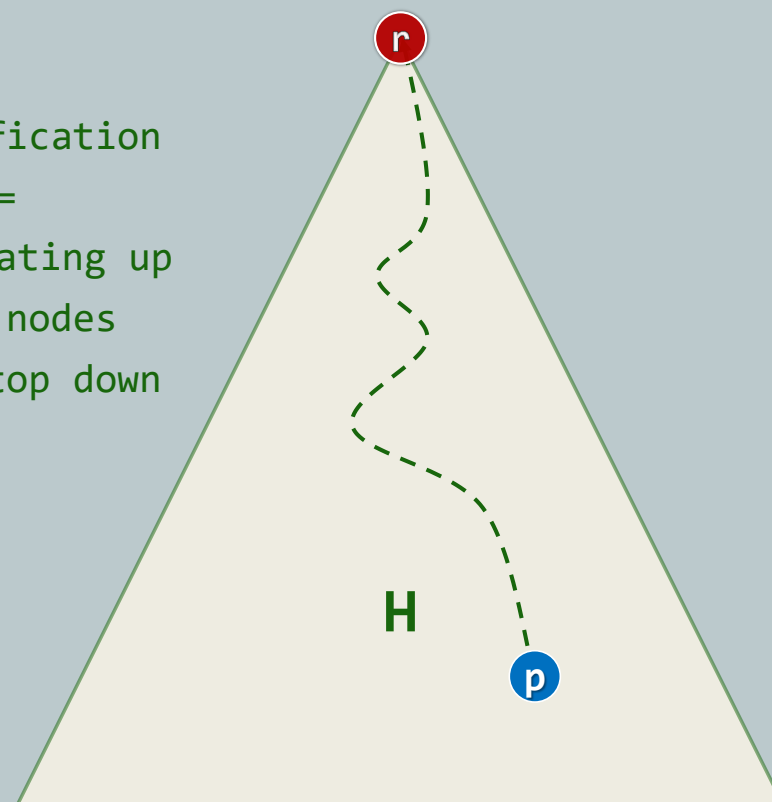
❖ 即便只考虑底层

- $n/2$ 个叶节点, 深度均为 $\mathcal{O}(\log n)$
- 累计耗时 $\mathcal{O}(n \log n)$

❖ 这样长的时间, 本足以全排序!

应该, 能够更快的...

heapification
=
percolating up
all nodes
from top down



自下而上的下滤

❖ 任意给定堆 $\mathcal{H}_0$ 和 $\mathcal{H}_1$, 以及节点 p

❖ 为得到堆 $\mathcal{H}_0 \cup \{p\} \cup \mathcal{H}_1$ 只需

将 r_0 和 r_1 当作 p 的孩子, 再对 p 下滤

❖ `template <typename T> //Robert Floyd, 1964`

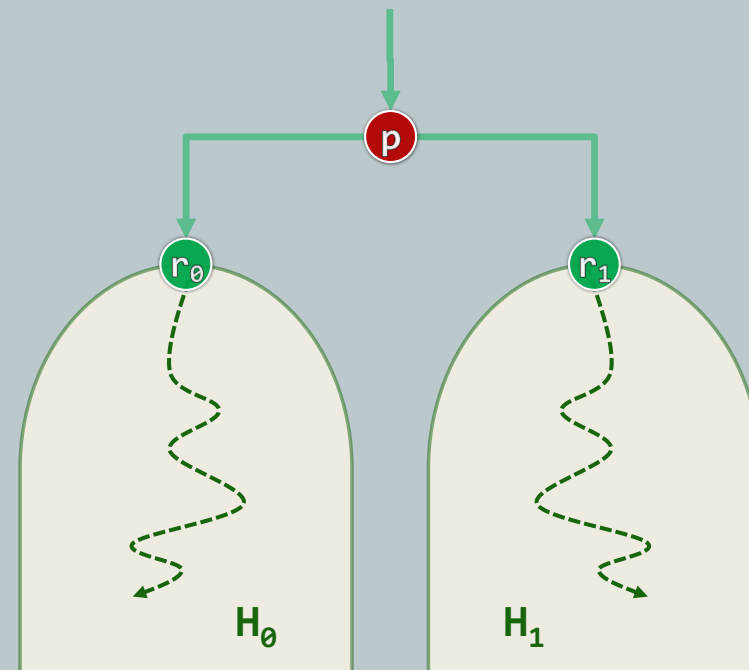
`void heapify( T* A, Rank n ) { //自下而上`

`for ( Rank i = n/2 - 1; 0 <= i; i-- ) //依次`

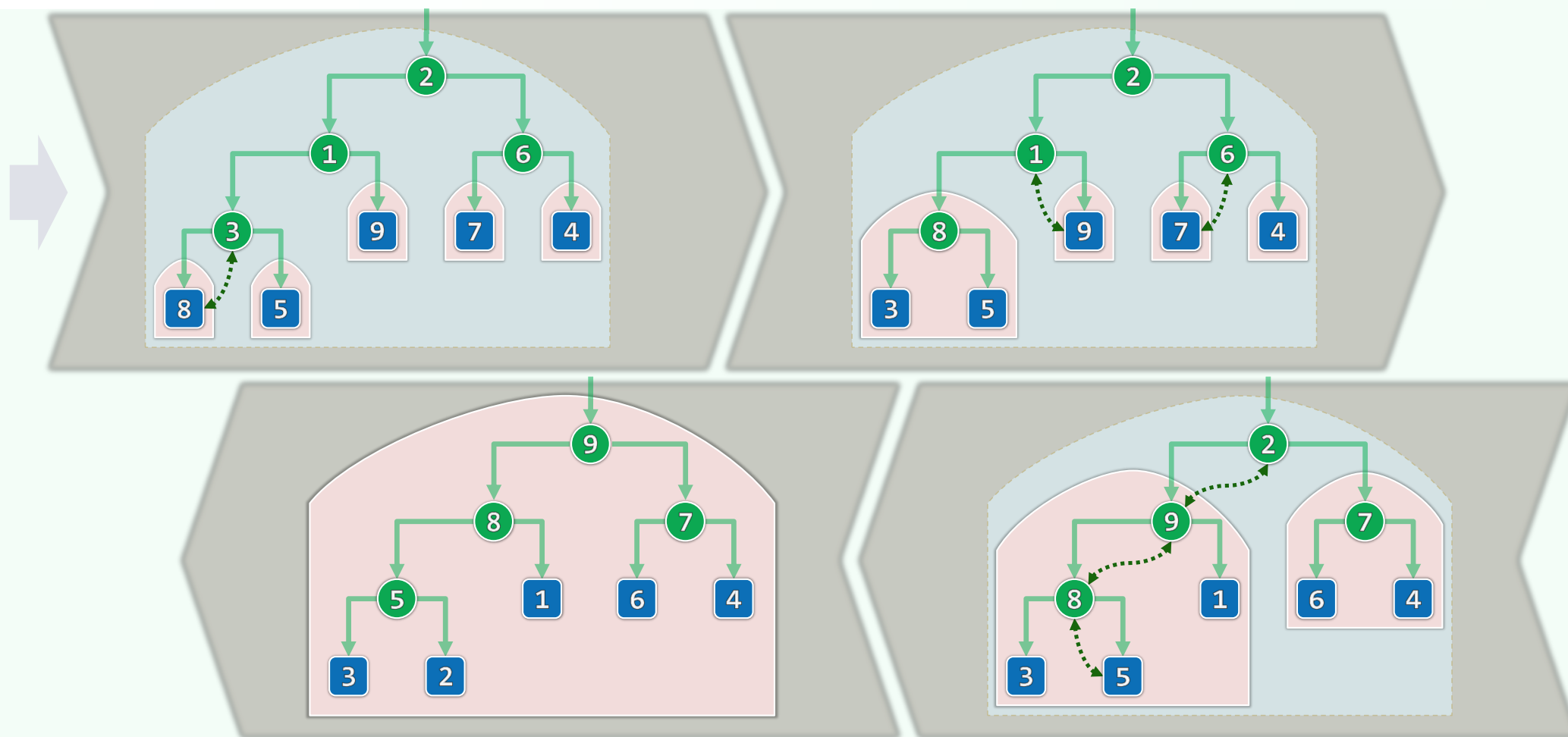
`percolateDown( A, n, i ); //下滤各内部节点`

`} //可理解为子堆的逐层合并`

`//由以上性质, 堆序性最终必然在全局恢复`



实例



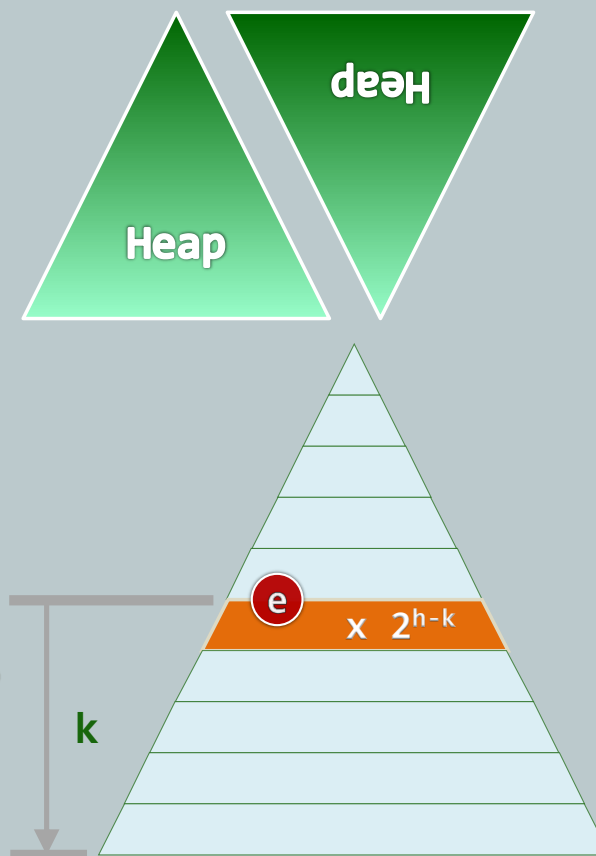
效率

❖ 每个内部节点所需的调整时间，正比于其**高度**而非**深度**

❖ 不失一般性，考查满树： $n = 2^{h+1} - 1$

❖ 所有节点的**高度**总和

$$\begin{aligned} S(n) &= \sum_{k=1}^h k \cdot 2^{h-k} = \sum_{k=1}^h \sum_{i=1}^k 2^{h-k} = \sum_{i=1}^h \sum_{k=i}^h 2^{h-k} \\ &= \sum_{i=1}^h \sum_{k=0}^{h-i} 2^k = \sum_{i=1}^h \{2^{h-i+1} - 1\} = \sum_{i=1}^h 2^{h-i+1} - h \\ &= \sum_{i=1}^h 2^i - h = 2^{h+1} - 2 - h = \mathcal{O}(n) \end{aligned}$$



优先级队列
多叉堆

多叉堆

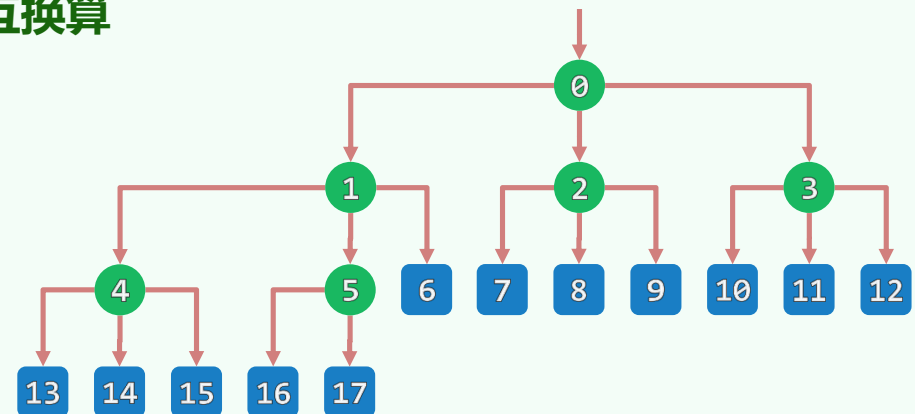
❖ 仍可基于向量实现，且父、子节点的秩可简明地相互换算

$$- \text{parent}(k) = \lfloor (k-1)/d \rfloor$$

$$- \text{child}(k, i) = k \cdot d + i, 0 < i \leq d$$

❖ 当然，d不再是2的幂时

将不再能够借助移位加速秩的换算



❖ heapify(): $O(n)$

不可能再快了（直接写入，亦不过如此）

❖ delMax(): $O(\log n)$



实质就是percolateDown() —— 已是极限了（为什么？）

❖ increase(): $O(\log n)$ —— 实质就是percolateUp() —— 似乎仍有改进空间...

优先级搜索

- ❖ 回顾图的PFS以及统一框架: $g \rightarrow \text{pfs}() \dots$
- ❖ 无论何种算法, 差异仅在于所采用的优先级更新器 $\text{prioUpdater}()$
 - Prim算法: $g \rightarrow \text{pfs}(0, \text{PrimPU}())$;
 - Dijkstra算法: $g \rightarrow \text{pfs}(0, \text{DijkstraPU}())$;
- ❖ 每一节点引入遍历树后, 都需要
 - 更新树外顶点的优先级 (数), 并
 - 选出新的优先级最高者
- ❖ 若采用邻接表, 两类操作的累计时间, 分别为 $O(n+e)$ 和 $O(n^2)$
- ❖ 能否更快呢?

优先级队列

❖ 自然地，PFS中的各顶点可组织为**优先级队列**

❖ 为此需要使用PQ接口

heapify(): 由n个顶点创建初始PQ

总计 $O(n)$

delMax(): 取优先级最高（极短）跨边(u, w)

总计 $O(n * \log n)$

increase(): 更新所有关联顶点到u的距离，提高优先级

总计 $O(e * \log n)$

❖ 总体运行时间 = $O((n+e)*\log n)$

- 对于**稀疏图**，处理效率很高
- 对于**稠密图**，反而不如常规实现的版本

❖ 有无更好的办法？

上山容易下山难

❖ 若将**二叉堆**改成**多叉堆** (d-heap) , 则堆高降至

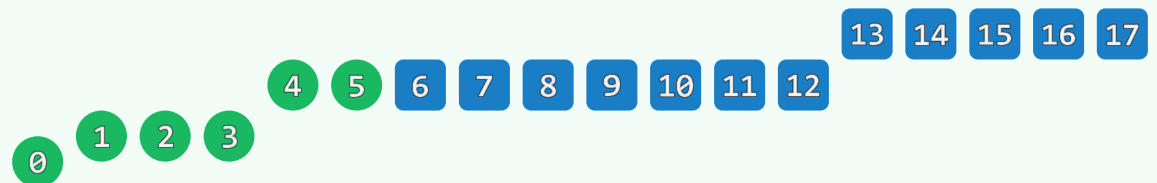
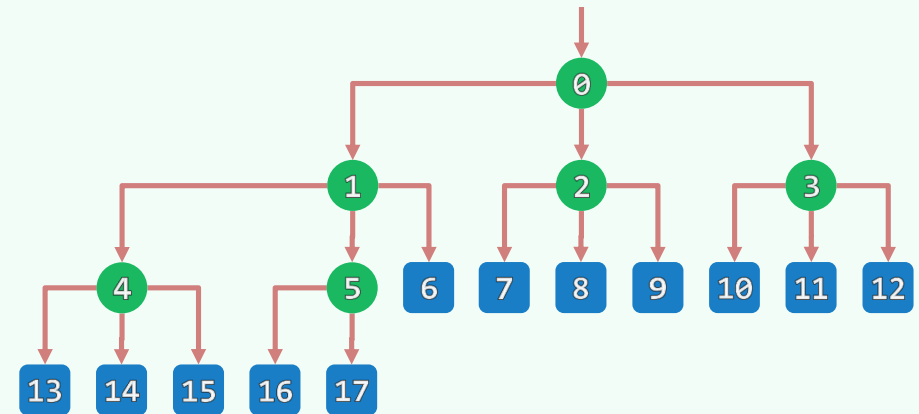
$$\mathcal{O}(\log_d n)$$

❖ 相应地, **上滤**成本降至

$$\log_d n$$

但 (只要 $d > 4$) **下滤**成本却增至

$$d \cdot \log_d n = \frac{d}{\ln d} \cdot \ln n$$



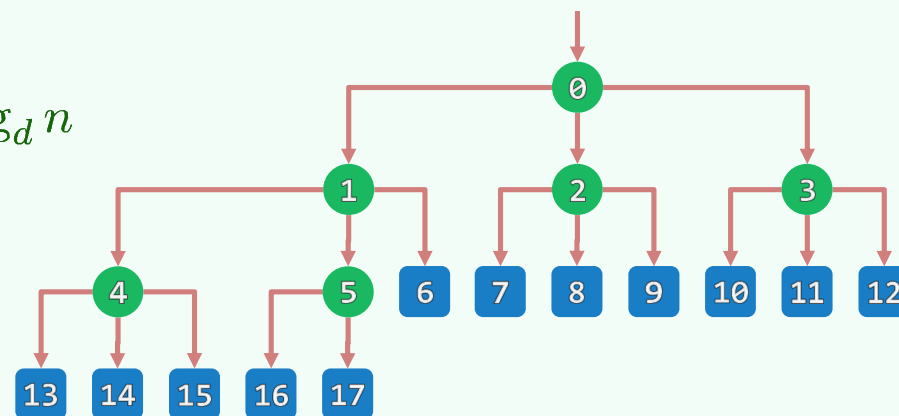
PFS

❖ 如此，PFS的运行时间将是：

$$n \cdot d \cdot \log_d n + e \cdot \log_d n = (n \cdot d + e) \cdot \log_d n$$

❖ 取 $d \approx e/n + 2$ 时

总体性能达到最优： $\mathcal{O}(e \cdot \log_{(e/n+2)} n)$



❖ 对于稀疏图保持高效：

$$e \cdot \log_{(e/n+2)} n \approx n \cdot \log_{(n/n+2)} n = \mathcal{O}(n \log n)$$

对于稠密图改进极大：



$$e \cdot \log_{(e/n+2)} n \approx n^2 \cdot \log_{(n^2/n+2)} n \approx n^2 = \mathcal{O}(e)$$

对于一般的图，会自适应地实现最优